第13章 模板

扫描全能王 创建

13.1 模板的概念

模板是什么？

template, 基于类型形参生成函数或类的一种机制。

也被称为“参量化类型”。

模板是代码重用的一种机制，使一个类/函数可操作多种类型的数据，而无需为每一种类型都设计一个类/函数。

有哪些模板？

- 函数模板，用于生成函数的模板，生成模板函数
- 类模板，用于生成类的模板，生成模板类
- 别名模板，用于生成类型别名的模板，C11

如何定义？

template < 类型形参 > *函数或类或别名的定义*

扫描全能王 创建

模板的类别

模板类别	定义	实例化结果	实例化方式
函数模板	带模板形参的函数： ● 非成员函数模板 ● 成员函数模板	函数 ● 非成员函数 ● 成员函数	隐式实例化：从实参推导模板实参 显式实例化：指定实参 显式特例化：指定实参并重定义函数体*
类模板	带模板形参的类 嵌套类模板	类 嵌套类	实例化：指定实参 显式特例化：指定实参并重定义类体*
别名模板*	带模板形参的类型别名	类型别名	指定实参

扫描全能王 创建

13.2 函数模板

函数设计中往往要对不同类型的数据进行相同的计算，就要为每一种类型都定义一个重载函数，例如：

```
int max(int x, int y){              return x>y ? x : y; }
double max(double x, double y){ return x>y ? x : y; }
float max(float x, float y){     return x>y ? x : y; }
```

注意到，对多种类型的数据处理有相同的计算过程，导致大量重复编码。

13.2.1 定义

定义一个“函数模板”，来生成所需要的函数：

```
template <class T>
    T max(T x, T y){ return x>y ? x : y; }
```

T为类型形参，可作为函数的形参类型、返回类型、内部变量的类型。

调用该函数模板，自动生成模板函数，并完成调用：

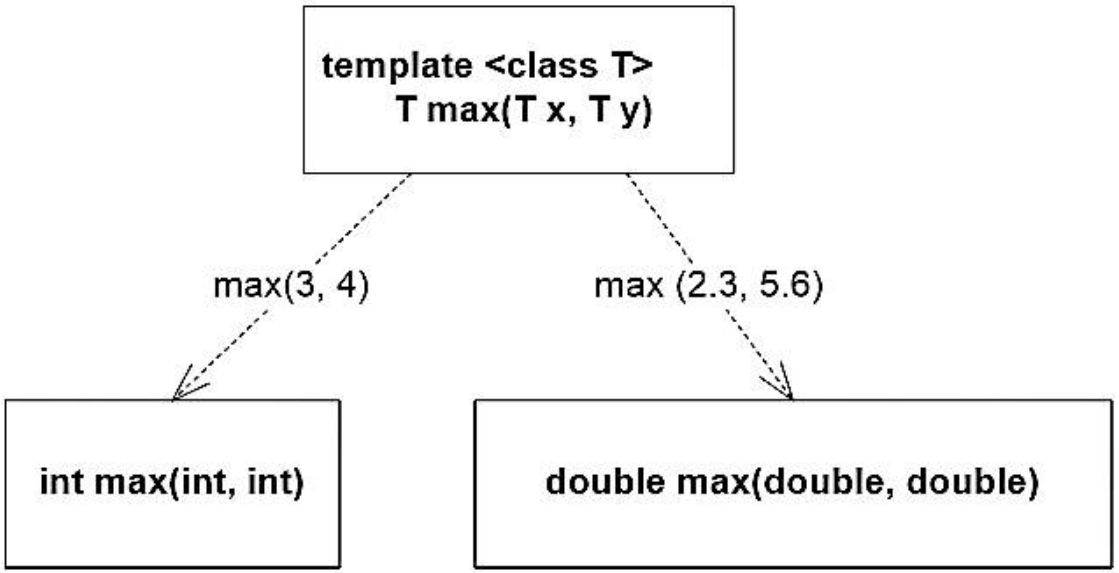
```
int i = 5, j = 6;
cout<<max(i, j); //注意：调用时无需显式指明实参的类型
```

编译时有2步，称为“模板函数的实例化”：

- 先把实参的类型作为模板的类型形参，生成一个模板函数。
- 检查实参类型与模板函数的一致性，传递实参的值完成模板函数的调用。

扫描全能王 创建

13.2.2 实例化



■以相同的过程处理不同类型的数据时，可定义函数模板。

■模板形参一般为大写单字母，以区别于普通形参。

■对于一个函数模板，在编译时根据实参类型自动替代模板形参，生成普通函数，再加以调用。

■要求实参类型能支持函数内实现所要求的运算。例如，**max**函数要求能支持“>”运算。

扫描全能王 创建

函数模板的例子

```
using namespace std;
#include <iostream>

template <class T>
T Tmax(T x, T y){return (x > y) ? x : y; }

int main(void){
  int x1 = 1, y1 = 2;
  double x2 = 3.4, y2 = 5.6;
  char x3 = 'a',y3 = 'b';
  cout<<Tmax(x1, y1)<< '\t';
  cout<<Tmax(x2, y2)<< '\t';
  cout<<Tmax(x3, y3)<<endl;
}
```

完整代码

扫描全能王 创建

13.2.3 函数模板与有参宏的区别

- 有参宏也能实现类似功能，例如：

#define max(x, y) (((x) > (y)) ? (x) : (y))

但存在区别：

1、在宏展开时编译器不能检查类型兼容性。如果实参对象不支持operator>运算，那么实参类型就不能进行宏展开，而宏展开时却不能给出错误信息。

2、注意到，两个宏参x或y将分别计算2次。如果调用方表达式中有自增或自减运算，那么就会执行两次，如：cout<<max(x1, ++y1);y1的结果为4，而不是预料的3。这是因为y1原值为2，在表达式中对y计算了2次。

3、由于宏扩展属于预处理，编译是针对宏展开之后的代码进行，编译错误只能给出宏展开之后的代码错误，而不是宏定义自身的错误。另外，在调试程序时也只能看到宏展开之后的形式。

扫描全能王 创建

13.2.4 函数模板的重载

函数模板之间可重载，而且与普通函数之间也可重载。

编译器通过如下匹配过程来确定调用哪一个函数。

1. 寻找和使用最符合函数名和参数类型的函数调用，即优先调用非模板的一般函数。
2. 如果找不到一般函数，则检查是否可用函数模板产生符合函数名和参数类型的模板函数，若有，则实例化。
3. 如果不能完全匹配，则检查是否可通过类型转换，按照步骤1和2再进行匹配。如果匹配，就调用它。
4. 如果上述操作均不能进行，则产生编译错误。如果最后得到了多于一种选择，那么这个函数调用有二义性错误。

扫描全能王 创建

```
#include<iostream>
#include<cstring>
//任意类型T的两个数据之间的交换
template <class T> void TSwap(T& a, T& b){
  T c = a;
  a = b;
  b = c;
}
//对T类型数组a元素升序排序
template <class T> void TSortArray(T*  a, int count){
    for(int i = 0; i < count-1; i++)
            for(int j = i + 1; j < count; j++)
                    if(a[i] > a[j])    //要求T支持operator >
                            TSwap(a[i], a[j]);
}
//打印T类型数组中的元素
template <class T> void TPrintArray(T *a, int count){
    for (int i = 0; i < count; i++)
            cout<<a[i]<<"   ";
    cout<<endl;
}
```

```
int main(){
 int intA[10] = {15,8,0,-6,2,39,-53,12,10,6};
 TSortArray(intA, 10);
 TPrintArray(intA, 10);

 double doubA[10] = {15.4,8.3,0,-
  6.2,2.7,39.5,-53.5,12.6,10.4,6.4};
 TSortArray(doubA, 10);
 TPrintArray(doubA, 10);

 char charA[18] = "CharSortTest";
 TSortArray(charA, strlen(charA));
 TPrintArray(charA, strlen(charA));
}
```

扫描全能王 创建

13.3 类模板

- 类模板就是用**template**说明的类，包含一个或多个类型形参。
- 在类模板中可用这些类型形参来说明数据成员的类型、构造函数或成员函数的形参或返回值，也可在函数体内使用。
- 类模板提供了强大的代码重用机制。
- 类模板的使用就是将类模板实例化为多个具体的类，即模板类，然后再通过模板类创建对象、操作对象。

扫描全能王 创建

13.3.1 定义

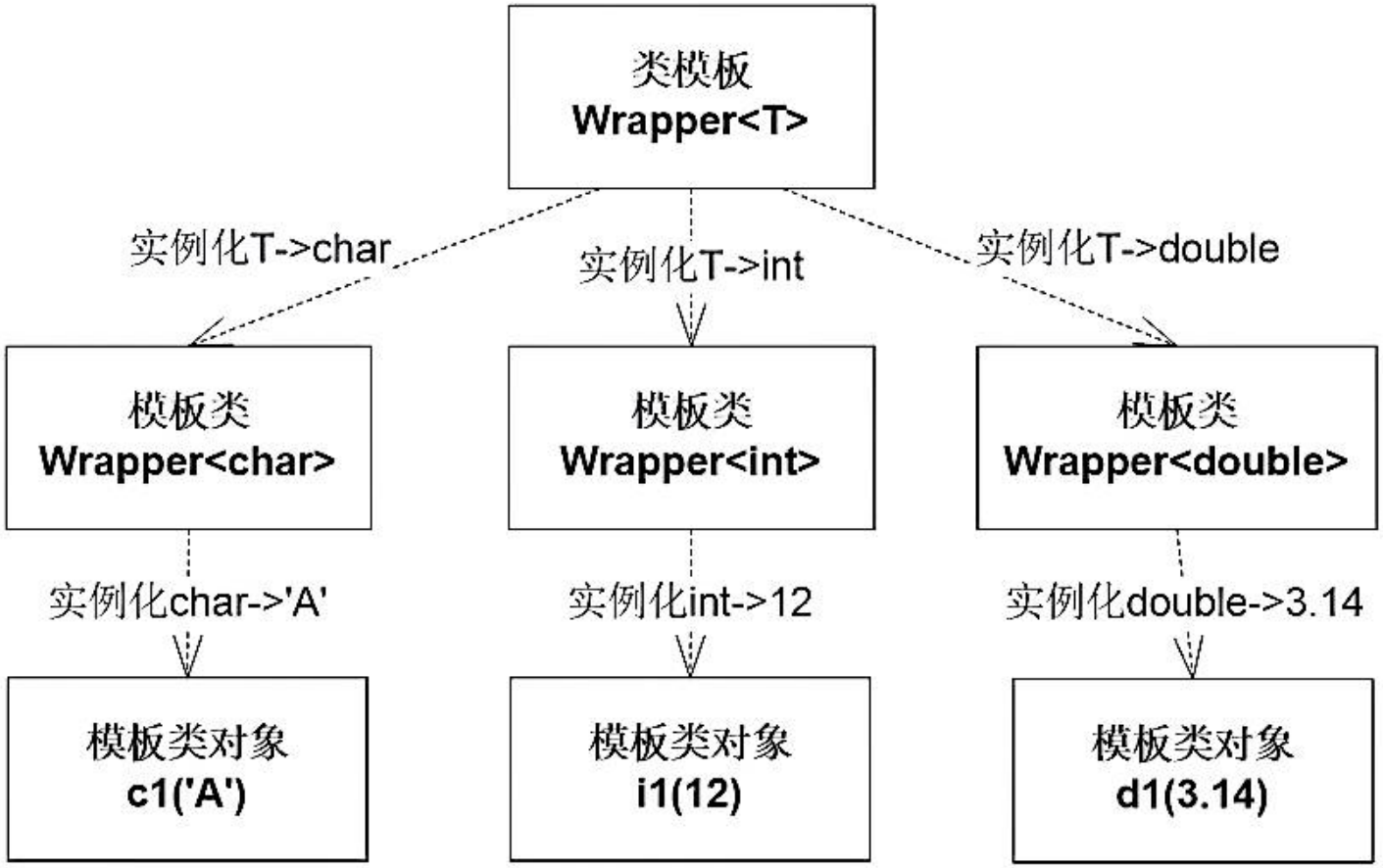
```
#include <iostream>

template<class T>
class Wrapper{
    T a;
public:
    Wrapper(const T a):a(a){}
    T get()const{return a;}
    void set(const T a);
};
template<class T>
void Wrapper<T>::set(const T a){this->a = a;}

int main(){
    Wrapper<char> c1('A');
    Wrapper<int> i1(12);
    Wrapper<double> d1(3.14);
    cout<<c1.get()<<endl;
    cout<<i1.get()<<endl;
    cout<<d1.get()<<endl;
}
```

扫描全能王 创建

13.3.2 类模板实例化



类模板的实例化

扫描全能王 创建

13.3.3 类模板的继承

类模板具有继承性，有4种形式：

- 普通类继承类模板；
- 类模板继承普通类；
- 类模板继承类模板；
- 类模板继承类模板，类型形参做基类

扫描全能王 创建

类模板的继承

1. 普通类继承类模板

基类是模板，派生类为非模板类。例如：

```
template<class T>
class TBase {
      T data;
public:
      TBase(const T a) :data(a) { cout << "TBase" << endl; }
      T get()const { return data; }
};
class Derived : public TBase<int> {
public:
      Derived(int a):TBase<int>(a) { cout << "Drived" << endl; }
};
```

基类是从类模板TBase实例化产生的一个模板类。

扫描全能王 创建

类模板的继承

2. 类模板继承普通类

基类为非模板类，派生类是模板。例如：

```
class TBase2{};
template<class T>
class TDerived2 : public TBase2 {
      T data;
public:
      TDerived2(const T a) :data(a) {
            cout << "TDrived" << endl; }
      T get()const { return data; }
};
```

派生类添加模板形参，成为类模板，其构造函数隐式调用基类的缺省构造函数。

扫描全能王 创建

类模板的继承

3.类模板继承类模板

基类与派生类都是模板，派生类模板应包含基类模板的类型形参，并可扩展新的类型参数。

```
#include<iostream>
using namespace std;
void f() {  cout << "global f()" << endl;}
template<typename T>
class A{
        T data;
public:
        A(T a) : data(a) {}
        void f() {cout << "A::f() "<<data<< endl;}
};
```

```
template<typename T>
class B :public A<T>{
        T data2;
public:
        B(T a, T b) :A<T>(a), data2(b) {}
        void g(){
                f();  //gloabl f(),调用的是全局函数
                this->f();  //A::f() ,调用的是成员函数
                A<T>::f(); //A::f()
        }
};
int main() {
        B<int> b(2,3);
        b.g();
}
```

扫描全能王 创建

类模板的继承

4.类模板继承形参类型

```
class BaseA{
public:
        BaseA(){ cout << "BaseA" << endl; }
};
class BaseB{
public:
        BaseB(){ cout << "BaseB" << endl; }
};
template<class T, int rows>
class BaseC{
private:
        T data;
public:
        BaseC() : data(rows) {
                cout << "BaseC:" << data << endl;
        }
        T get() { return data; }
};
```

```
template<class T>
class Derived : public T { //A类型形参做基类
public:
        Derived():T()
    { cout << "Derived" << endl; }
};
int main(){
        Derived<BaseA> x;    //BaseA作为基类
        Derived<BaseB> y;    //BaseB作为基类
        Derived<BaseC<int, 3>> z;
        //BaseC<int,3>作为基类
        cout << z.get() << endl;
        return 0;
}
```

扫描全能王 创建

[例] 定义一个通用的栈类

```
template <class T, int n = 10>
class AnyStack
{   T  m_tStack[n];
    int  m_nMaxElement;
    int  m_nTop;
    public:
        AnyStack() : m_nMaxElement(n), m_nTop(0){}
        int  GetTop() {return m_nTop;}
        bool Push(T);//入栈函数
        bool Pop(T&);//出栈函数
};
```

```
template <class T, int n>
bool AnyStack <T, n>::Pop(T &elem)
{  if(m_nTop > 0)
   {  m_nTop--;
      elem = m_tStack[m_nTop];
      return true;
   }
   else
      return false;
}
template <class T, int n>
bool AnyStack <T, n>::Push(T elem)
{  if(m_nTop<=m_nMaxElement)
   {  m_tStack[m_nTop] = elem;
    m_nTop++;
      return true;
   }
   else
    return false;
}
```


扫描全能王 创建

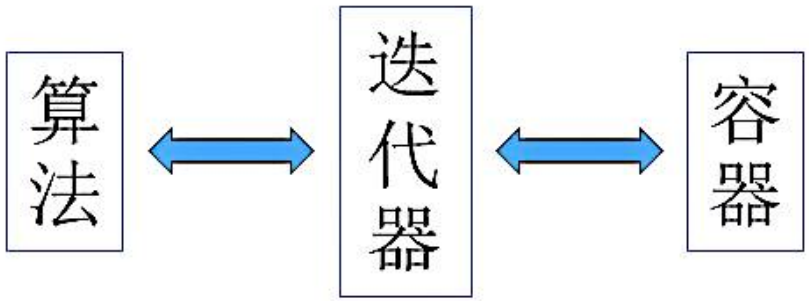
[例] 定义一个通用的栈类(续)

```
int main()
{  int n;        char * s1;
   AnyStack <int，20> iStack;//定义一个整数栈
   iStack.Push(5);       iStack.Push(6);    iStack.Pop(n);
   cout << "第一个出栈整数=" << n << endl;
   iStack.Pop(n);
   cout << "第二个出栈整数=" << n << endl;
   AnyStack <char *> strStack;//定义一个字符串栈
   strStack.Push("It's first string");
   strStack.Push("It's second string");
   strStack.Pop(s1);    cout << "第一个出栈字符串=" << s1 << endl;
   strStack.Pop(s1);    cout << "第一个出栈字符串=" << s1 << endl;
   return 0;
}
```


扫描全能王 创建

13.4 标准模板库STL

- STL，Standard Template Library
- STL模板库最大优势是可重用(reusable)
- STL成为ANSI/ISO C++标准的一部分；
- 主要包括容器container，迭代器iterator与算法algorithm



扫描全能王 创建

13.4.1 容器container

一组类模板，用来管理多个元素的序列。一个容器中的多个元素都具有相同类型

- 序列容器(sequence container)
- 关联容器(associative container)

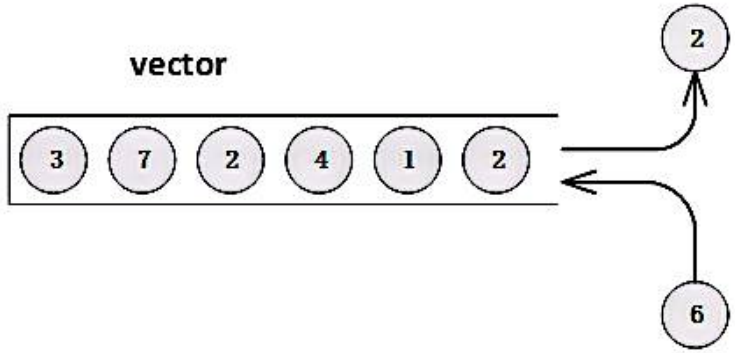
  前两者合称头等first class容器。
- 容器适配器(container adapter)

  基于头等容器的定制，如stack，queue
- 仿容器(near container)

  string是basic_string的别名类型

  array，valarray，bitset等

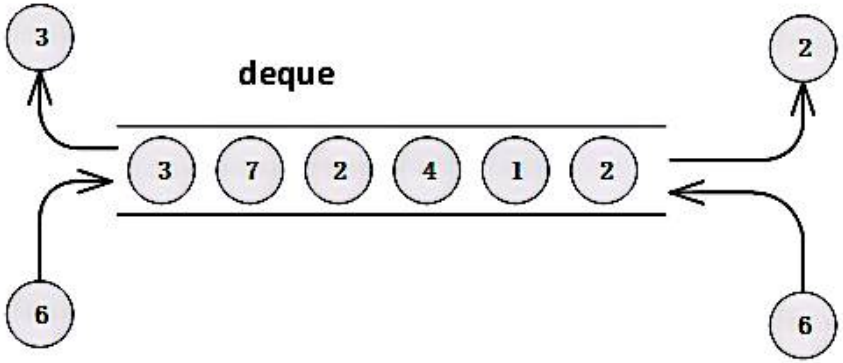
扫描全能王 创建

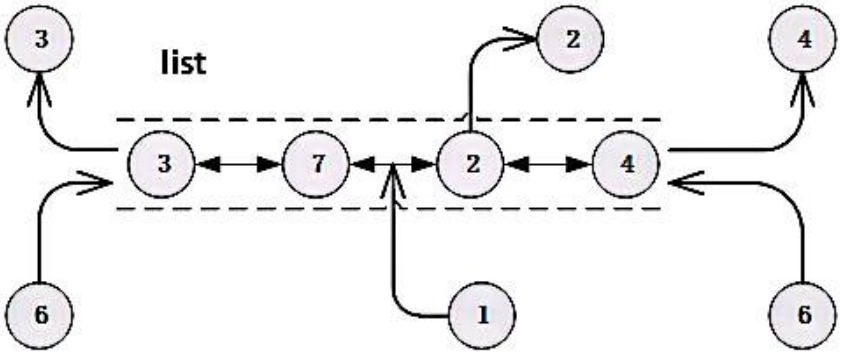


3种序列容器

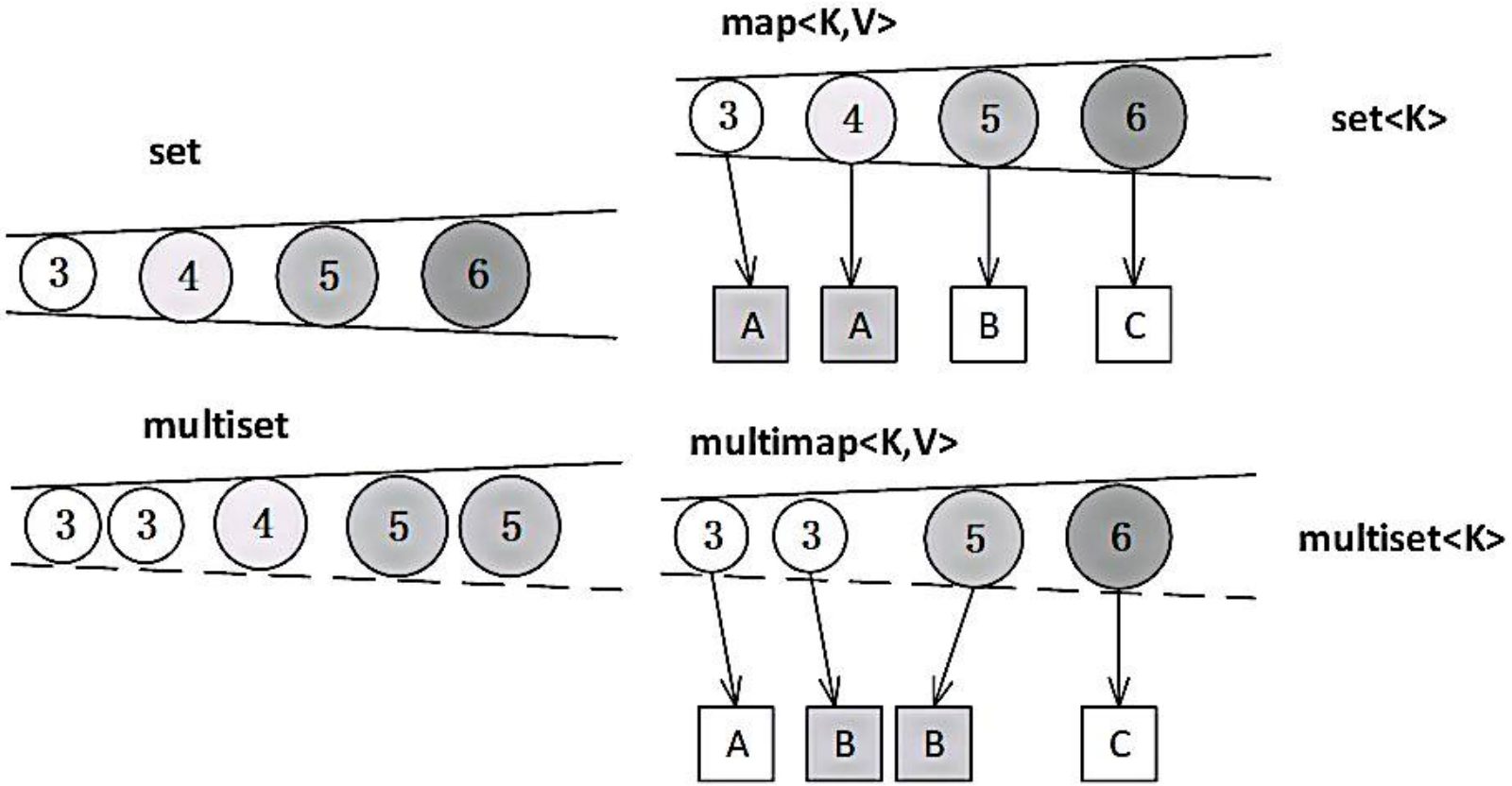
元素间次序和位置

操作元素的方式





扫描全能王 创建



4种关联容器

扫描全能王 创建

容器名	中文名	特点	头文件	备注
vector<T>	向量	适合在序列尾端加入或删除元素。 适合随机访问各元素	<vector>	随机访问按下标，下标范围为[0–size()-1] 不适合中间插入或删除
deque<T>	双端队列	适合在序列两端加入或删除元素。 适合随机访问各元素	<deque>	全称double-ended queue 不适合中间插入或删除
list<T>	表，链表	适合在序列中间插入或删除元素。 适合双向遍历元素	<list>	不适合随机访问。
set<T>	集合	各元素不重复，升序，键值 适合双向遍历	<set>	能支持 operator== operator< 不适合随机访问。
multiset<T>	多集	可重复元素，升序 可双向遍历元素	<set>	元素按值升序排序。各元素值不唯一，同值元素相邻。 不适合随机访问。
map<K,V>	映射， 单值映射， 单射	元素是对偶pair<key, value> 一对一、多对一 键值不重复。适合双向遍历	<map>	各元素按键值升序排列。 各元素的键是一个集合。
multimap<K,V>	多射， 多值映射	元素是对偶pair<key, value> 一对多、多对多 键可重复。适合双向遍历	<map>	各元素按键值升序排列。 各元素的键是一个多集。

扫描全能王 创建

如何选择合适的容器？

- 识别你要处理的一组元素/对象/值的共同类型。
- 元素如何进入容器？前端、后端、中间插入？
- 元素如何被访问？随机访问、遍历访问？
- 元素是否需要交换位置？出于某种排序的目的
- 元素之间是否具有显式的次序要求？
- 元素之间是否存在唯一性的、不重复的标识？
- 元素之间是否可重复？
- 元素如何离开容器？前端、后端、中间离开？
- 一个元素是单值，还是<键-值>对？
- 如果元素是<键-值>对，其中的键是否允许重复？

扫描全能王 创建

13.4.2 迭代器iterator

迭代器是用来访问容器中元素的一种数据结构，就是指针的抽象，但更安全，功能更强。

如何获得迭代器？从已有容器中读取迭代器。

设有vector<int> vint; 那么vint.begin()返回一个迭代器，指向容器vint中的第一个元素，vint.end()返回迭代器指向最后一个元素后面的空位置。

一个容器c中元素位置范围就是[c.begin(), c.end())

迭代器如何操作？

设it是一个迭代器，指向某个元素，那么*it就是迭代器所指向的元素。

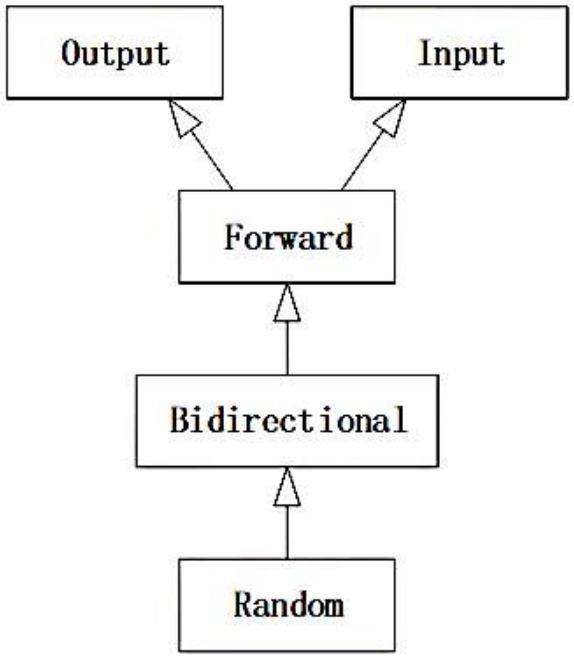
++it和it++就是后移指向下一个元素，--it和it--就是前移指向前一个元素。

扫描全能王 创建

迭代器的作用和类型

1、用来访问容器中的各个元素，随机访问或按顺序访问。

2、作为STL算法与容器之间的中介，使STL算法能作用于容器中的元素。



六大类
继承关系反映可替代关系
下方可替代上方；
当需要某类迭代器，可提供其下方类型的迭代器

扫描全能王 创建

类别	形参名	功能	备注
Output 输出	OutIt	向容器中写入元素。迭代器X仅对应一个值V，写入过程相当于*X++ = V。容器中添加了一个元素。	这里的输出是将元素写入容器，只能从前向后移动，而且只能遍历一遍。
Input 输入	InIt	从容器中读出元素。如果迭代器不等于end()，就指定了一个元素，可以多次读取，V = *X。要读取下一个元素，就要递增，V = *X++。容器中元素不变。	输入是从容器中读出元素。 迭代器只能从前向后移动，而且只能遍历一遍。 一类容器都持有const_iterator和const_reverse_iterator分别用来正向和逆向读取元素。
Forward 正向	FwdIt	正向迭代器可取代输出迭代器用于写入元素，也可替代输入迭代器用于读出元素。用V=*X读出刚用*V=X写入的元素。可用多个迭代器独立操作。	一类容器都持有iterator类型，作为正向迭代器，从begin()到end()之前，自增后移，自减前移。
Bidirectional 双向	BidIt	双向迭代器X可取代正向迭代器，而且增加了前移操作，V = *X--	一类容器都持有reverse_iterator类型，作为逆向迭代器，从rbegin()到rend()之前，自增前移，自减后移。
Random 随机访问	RanIt	随机访问迭代器可取代双向迭代器，而且增加了用整数作为下标，就像数组下标，如N是一个整数，那么x[N], x+N, x-N	只有vector和deque支持随机访问

扫描全能王 创建

从2023年开始电瓶车违章纳入交警监控记录。为配合交警对违章记录的操作，请编写程序实现输入n次违章车辆的车牌号，统计各车牌违章次数并按违章次数降序输出。

(1) 先输入n(2<=n<=100)，再输入n次违章车牌号（可重复），车牌号中间没有空格，重复出现的车牌号表示该车多次违章。根据输入的车牌号，统计各车牌的违章次数。

(2) 按各车牌的违章次数**降序排序**，并输出各车牌及其违章次数。如果多个车牌的违章次数一样，那么这些车牌号次序无所谓。输出时1个车牌号占1行。

例如，输入：

4
苏A55555
苏A33333
苏A22222
苏A33333

输出：

车牌号:违章次数(降序)
苏A33333:2
苏A55555:1
苏A22222:1

```
#include<map>
#include<iostream>
using namespace std;
int main(){
   int n;
            cin >> n;
            map<string,int> a;
            for(int i = 0; i < n; i++){
                        string x;
                        cin >> x;
                        if(a.find(x) != a.end()) a[x]++;
                        else a[x] = 1;
            }
            multimap<int,string> b;//次数(升序):车号
            for(auto e : a) b.emplace(e.second, e.first);
            cout<<"车牌号:违章次数(降序)"<<endl;
            for(auto rit = b.rbegin(); rit != b.rend(); rit++)
               cout<<rit->second<<":"<<rit->first<<endl;
}
```

扫描全能王 创建

先输入n，再输入n个原始数据，含商品名、种类和数量，用空格分开，计算并输出各种类的各商品名的累计数量。

例如，

输入：

5
草莓 水果 2
白菜 蔬菜 3
车厘子 水果 3
白菜 蔬菜 4
草莓 水果 5

输出：

蔬菜
白菜 7
水果
草莓 7
车厘子 3

```
#include<iostream>
#include<map>
#include<string>
using namespace std;
int main(){
        map<string,map<string,int>> res;
        int n; cin>>n;
        while(n--){
                string name; cin>>name;
                string kind; cin>>kind;
                int num; cin>>num;
                if(res.find(kind) == res.end()){//新类别
                        map<string,int> skill;
                        skill[name] = num;
                        res[kind] = skill;
                }else{ //已有类别
                        auto eskill = res[kind]; //取出已有商品
                        if(eskill.find(name) == eskill.end()){ //查找品名
                                eskill[name] = num;
                        }else{ //已有商品名，添加计数
                                eskill[name] = eskill[name] + num;
                        }
                        res[kind] = eskill;
                }
        }
        for(auto x : res){
                cout<<x.first<<endl;
                for(auto y : x.second)
                        cout<<"    "<<y.first<<' '<<y.second<<endl;
        }
}
```

扫描全能王 创建

13.4.4算法<algorithm>

- 85个容器相关的函数模板，如sort、find、copy等。
- 算法具有通用性，可作用于多种STL容器，也适用于数组、自定义类型。
- 算法通过迭代器与被操作容器关联
- 函数形参往往就是某种迭代器。

扫描全能王 创建

表 13-11 算法分类<algorithm>

1、不改变操作对象内容
通用判断：all_of; any_of; none_of;
线性查找: find; find_if; find_if_not; find_end; adjacent_find; find_first_of
子序列匹配: search; search_n
计数: count; count_if
遍历: for_each
比较两个区间: equal; equal_range; mismatch; lexicographical_compare
最大与最小值: min; max; min_element; max_element; minmax; minmax_element
2、改变操作对象内容
复制区间: copy; copy_backward; copy_if; copy_n
互换元素: swap; iter_swap; swap_ranges
转换：transform
替换: replace; replace_if; replace_copy; replace_copy_if
生成与填充: fill; fill_n; generate; generate_n
移动：move; move_backward
移除元素: remove; remove_if; remove_copy; remove_copy_if; unique; unique_copy
逆序、轮转、排列: reverse; reverse_copy; rotate; rotate_copy; is_permutation; next_permutation; prev_permutation.
分割: is_partitioned; partition; partition_copy; partition_point; stable_partition
随机重排: shuffle; random_shuffle*
3、高级功能
排序: sort; is_sorted; is_sorted_until; stable_sort; partial_sort; partial_sort_copy; nth_element;
二分查找: binary_search; lower_bound; upper_bound;
合并: merge; inplace_merge

扫描全能王 创建

```
#include <vector>
#include <algorithm>
#include <iostream>
#include <random>
using namespace std;
bool greater5(int value) {                                    //A 一元谓词函数
    return value > 5;
}
int main() {
    vector <int> v1;
    vector <int>::iterator Iter1;                             //B 迭代器变量
    for (int i = 0; i <= 10; i++)
        v1.push_back(i);
    random_device rd;                                 //说明一个 URNG 设备 rd
    mt19937 gen(rd());                                //说明一个随机数发生器，并用 rd 初始化
    shuffle(v1.begin(),v1.end(),gen);                 //第 3 个实参是随机数发生器
    cout << "Vector v1 is ( ";
    for (Iter1 = v1.begin(); Iter1 != v1.end(); Iter1++)//C 用迭代器遍历
        cout << *Iter1 << " ";
    cout << ")." << endl;
    // Partition the range with predicate greater5
    partition(v1.begin(), v1.end(), greater5);                //D 第 3 个形参
    cout << "The partitioned set of elements in v1 is: ( ";
    for (Iter1 = v1.begin(); Iter1 != v1.end(); Iter1++)   //E 再次遍历
        cout << *Iter1 << " ";
    cout << ")." << endl;
    return 0;
}
```

partition：为对指定范围内元素重新排序，使用输入的函数，把结果为true的元素放在结果为false的元素之前

扫描全能王 创建

命名空间namespace

- 命名空间(namespace)是解决大程序中多个元素(如类、全局函数、全局变量等)命名冲突的一种机制。命名空间结构类似文件系统中的目录或文件夹，空间中的成员就类似目录中的文件。全局空间相当根目录，main函数只能定义在全局空间中。一个目录名作为其中多个不重名文件的命名空间，子目录作为嵌套子空间，文件作为该空间中的成员，同时命名空间也对其成员提供作用域。

- 命名空间的基本规则如下：
  - 一个程序所用的多个命名空间在相同层次上不重名；
  - 在同一个命名空间内的所有成员名字不重复；
  - 在一个命名空间中可嵌套定义其内层的多个子空间。

- 定义命名空间的语法格式如下：
- namespace [<标识符>]{一组成员}

扫描全能王 创建

命名空间namespace

```
namespace spaceA{
  int m,n;
  void fun1(){m+=5;}
  void fun2(){fun1();}
}   //没有“;”号
void main(){
  spaceA::m=12;
  spaceA::fun1();
}
```


扫描全能王 创建

命名空间namespace

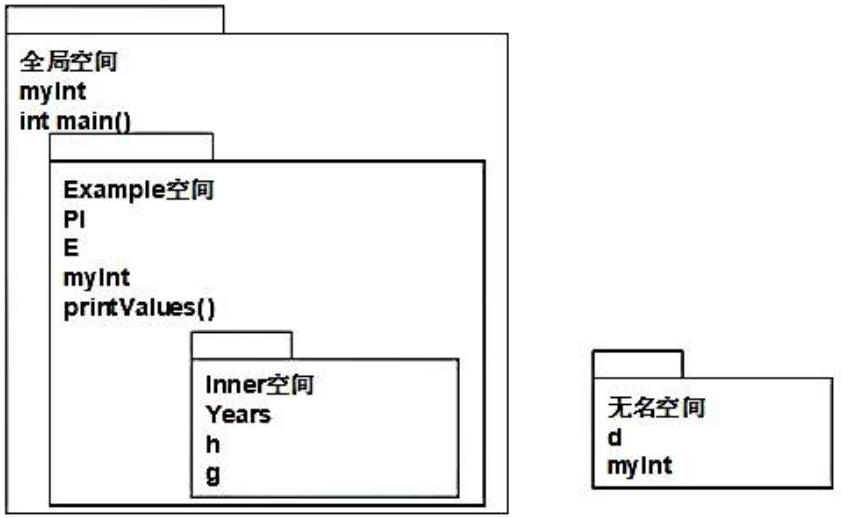
- 可以看出，在命名空间外面对命名空间中的变量与函数访问时，需加上命名空间作用域限定符。命名空间结束处不应该加分号，一个命名空间可以分散在多个文件中。
- 在命名空间引用到命名空间的变量时，都要加上命名空间前缀，为了解决这件令人感到麻烦的事，C ++引入如下语句：

  using namespace 命名空间名；

扫描全能王 创建

```
#include<iostream>
using namespace std;
int myInt = 98;      //全局命名空间
namespace Example{                                          //定义命名空间Example
          const double PI = 3.14159;
          const double E = 2.71828;
          int myInt = 8;
          void printValues();
          namespace Inner{
                    enum Years{FISCAL1 = 2005, FISCAL2, FISCAL3};
                    int h = ::myInt;
                    int g = myInt;
          }
}
namespace{                        //定义无名空间
          double d = 88.22;
          int myInt = 45;
}

int main(){          //main只能在全局空间中
          cout<<"In main";
          cout<<"\n(global) myInt = "<<::myInt;
          cout<<"\nd = "<<d;
          cout<<"\nExample::PI = "<<Example::PI
                    <<"\nExample::E = "<<Example::E<<"\nExample::myInt = "<<Example::myInt
                    <<"\nExample::Inner::FISCAL3 = "<<Example::Inner::FISCAL3
                    <<"\nExample::Inner::h = "<<Example::Inner::h<<"\nExample::Inner::g = "<<Example::Inner::g<<'\n';
          Example::printValues();
          return 0;
}
void Example::printValues(){
          cout<<"\nIn Example::printValues:\n"<<"myInt = "<<myInt<<"\nd = "<<d<<"\nPI = "<<PI<<"\nE = "<<E
                    <<"\n::myInt = "<<::myInt     <<"\nInner::h = "<<Inner::h
                    <<"\nExample::Inner::g = "<<Example::Inner::g<<endl;
}
```



扫描全能王 创建

命名空间namespace

◆事实上，目前C ++标准中的标准类库的变量与函数都属于命名空间std，如cout、cin等，由于相应的头文件中都有语句“using namespace std;”，对上述变量的引用无需再写成：std::cout和std::cin。

◆新的C ++标准引入了一种新的头文件载入方式： 没有.h。

扫描全能王 创建

C ++标准类库的头文件名称

<algorithm> <bitset> <complex> <deque>
<exception> <fstream> <functional> <iomanip>
<ios> <iosfwd> <iostream> <istream>
<iterator> <list> <locate> <limits>
<map> <memory> <new> <numeric>
<ostream> <queue> <streambuf> <string>
<set> <sstream> <stack> <stdexcept>
<typeinfo> <utility> <valarray> <vector>
<cmath>

扫描全能王 创建

std

◆C++标准引入的std命名空间并不向后兼容老的C++标准库。

◆老的C++标准库的头文件中声明的标识符是全局范围的，不需要使用std命名空间限定就可以使用。

◆为了区分在标准化进程中的这种变化，C++新的标准库启用了新的头文件命名格式。这样就允许程序员通过包含不同格式的头文件来使用不同的c++标准库。

◆新的C++标准库的头文件不再包含扩展名(.h, .hpp, .hxx等），形式如下：

```
#include <iostream>
#include <string>
```


扫描全能王 创建

std

•这种新标准可以同样涵盖到C标准库，C标准库的头文件现在可以这样引用：

```
#include <cstdlib> //was: <stdlib.h>
#include <cstring> //was: <string.h>
```

而这种新格式的头文件中定义的标识符全部涵盖在std命名空间下。

•这种新的命名方式的便利之处就在于可以方便地区分老的C标准库中的头文件和新的C++标准库中的头文件。比如C标准库中和C++标准库中原先都有一个<string.h>的头文件，如果同时使用的话，会很不方便。现在就不会存在这样的问题了，形式如下：

```
#include <string> //C++ string class
#include <cstring> //C char* functions
```


扫描全能王 创建

std

注意：

- 这种新的命名标准并不是意味着在文件系统中标准库的头文件没有扩展名，标准库的头文件在实际的文件系统中可以有扩展名，也可以没有扩展名，这取决于不同的C++系统include语句的实现。
- 在实践中，大部分的C++系统的include语句就是简单地到文件系统中查找相同名称的头文件，然后加载它，所以大部分c++系统的标准库头文件就是直接没有扩展名。
- 但是，这种规则仅仅应用在标准库的头文件中，通常来说对于程序员自己定义的头文件，还是加上特定的扩展名比较好。

扫描全能王 创建

std

- 当然，为了兼容性，原先C标准库的头文件还可以使用，诸如：

```
#include <stdlib.h>
```

这种情况下，所对应的标识符（变量或这函数）同时在全局范围和std命名空间中声明。

- 而对于C++标准库的老的头文件格式的使用，诸如：

```
#include <iostream.h>
```

在现有C++标准中没有说明，因此在理论上不应该被支持了。但是在实际中，一些C++系统的厂商实现中却提供了老的头文件格式，是为了向后兼容。

- 在实际使用中，程序员应该根据自己的情况选择一种情况来使用，尽量使用新标准的格式，但如果为了兼容性一定要使用老的文件格式，也不是不可以。

扫描全能王 创建

异常处理机制

- C ++的异常处理使程序可以向更高的层次传递意想不到的事件，从而使程序能更好地从这些异常事件中恢复过来。

扫描全能王 创建

异常处理机制

1. 异常处理的语法

① throw表达式用来检测是否产生异常，若是，则抛掷异常。语法如下：

throw 表达式;

② try 是一个复合语句块，它将那些有可能产生异常的语句框定在try块中，并根据异常的情况使用不同的throw表达式抛出异常。语法如下：

```
try{//复合语句
        ……
        throw 表达式1;
        ……
        throw 表达式2;
        ……
        throw 表达式n;}
```

③ 紧接try块，使用catch块，将异常处理的语句放在catch块中，以便异常被传递过来时就处理它。每个catch块是一个复合语句，处理相应的异常。语法：

```
catch(异常类型说明1)
 {……//复合语句}
catch(异常类型说明2)
{……//复合语句}
……
catch(异常类型说明n)
{……//复合语句}
```


扫描全能王 创建

异常处理机制

- 如果某程序中发现了自己不能处理的异常，就可以使用throw表达式抛掷这个异常，将它抛掷给调用者。
- throw的操作数在表示异常类型语法上与return语句的操作数相似，如果程序中有多处要抛掷异常，应该用不同的操作数类型来相互区别，操作数的值不能用来区别不同的异常。

扫描全能王 创建

异常处理机制

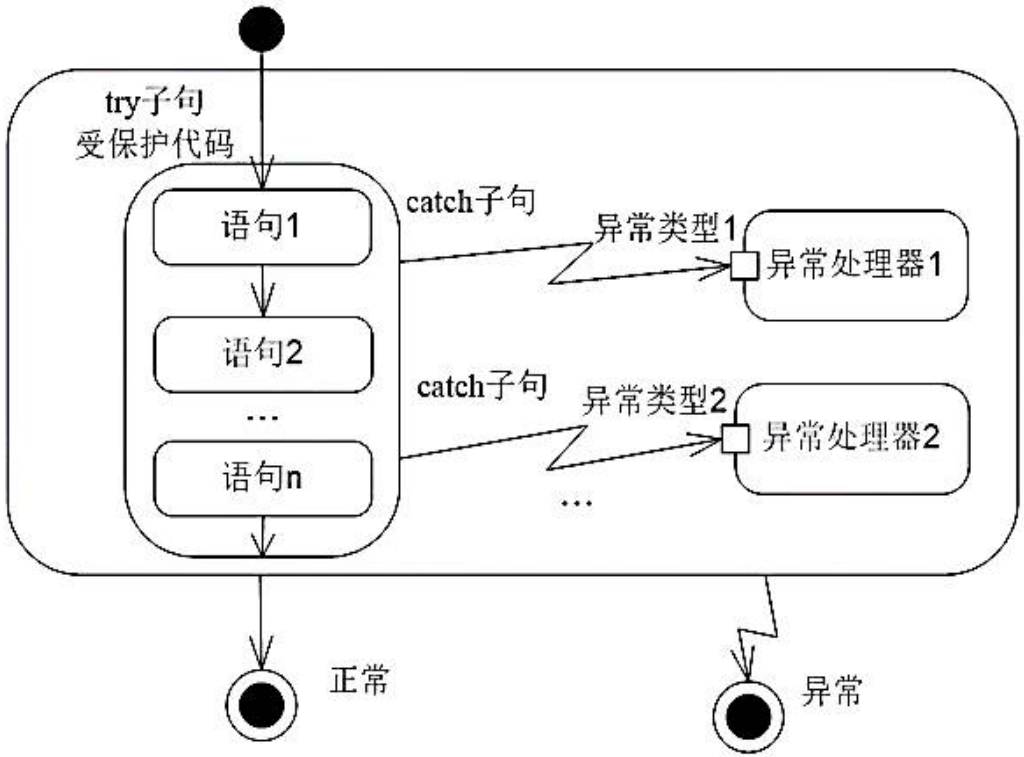
- try块中的复合语句是代码的保护段。如果预料某段程序代码（或对某个函数的调用）有可能发生异常，就将它放在try块中。
- 如果这段代码（或被调用的函数）运行时真的遇到异常的情况，其中的throw表达式就会抛掷这个异常。
- 紧接try块的catch块复合语句是异常处理程序，“捕获”由throw表达式抛掷的异常。

扫描全能王 创建

异常处理的执行过程

① 控制通过正常的顺序执行到达try语句，然后执行try块的保护段。

② 如在保护段执行期间没有引起异常，就跳过try块后的所有catch块，继续执行紧接最后一个catch块后面的程序。



扫描全能王 创建

异常规格说明

- 异常规格说明存在于函数说明中，位于参数列表之后。函数的所有潜在异常类型均随着关键字throw而插入函数说明中。如：
  - void SetElem(int i, int j, double val) throw(InvalidIndex);
- 而传统的函数声明，如：

  void SetElem(int i, int j, double val);

  可能抛出任何一种异常。
- 好的编程习惯：对于每一个有异常抛出的函数都应当加入异常规格说明。

扫描全能王 创建

```
int main()
{
        Point point;
        point.x=0;
        point.y=0;
        try
        {
                f(1);
                f(point);
        }
        catch(int e)
        {
                cout<<"int异常:"<<e<<endl;
        }
        catch(Point e)
        {
                cout<<"Point异常:("<<e.x<<","<<e.y<<")"<<endl;
        }
        return 0;
}
```

```
int 异常:1
```

```
#include <iostream.h>
struct Point{
        int x;
        int y;
};
void f(int n)
{
        throw 1;
}
void f(Point point)
{
        Point p;
        p.x=0;
        p.y=0;
        throw p;
}
```


扫描全能王 创建

```
int main()
{
        double x,y;
        cout<<"输入两个数： ";
        while(cin>>x>>y)
        {
                try
                {
                        cout<<hmean(x,y)<<endl;
                }
                catch(char *p)
                {
                        cout<<p<<endl;
                }
                cout<<"继续输入数值<q退出>:";
        }
        return 0;
}
```

```
#include <iostream>
using namespace std;
double hmean(double a,double b)
{
        if(a==-b)
                throw "zero divide error";
        double z=2.0*a*b/(a+b);
        return z;
}
```

```
输入两个数： 10 20
13.3333
继续输入数值<q退出>:10 -10
zero divide error
继续输入数值<q退出>:q
```


扫描全能王 创建

实例

- 除数为0异常处理示例。在整数除法中，如果除数为0就引发底层未命名异常，因此有必要在除法执行之前判断除数是否为0，如果除数为0就引发一个命名的异常来通知调用方。

扫描全能王 创建

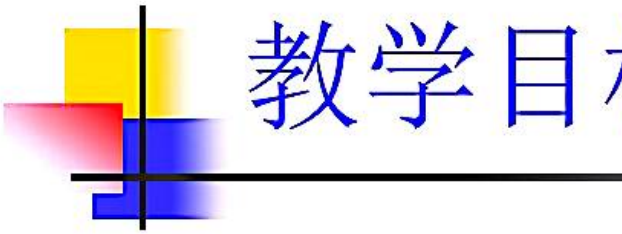
```
#include <iostream>
#include <stdexcept>
using namespace std;
double quotient(int numrator, int denominator) throw(invalid_argument){
        if (denominator == 0)
                throw invalid_argument("denominator is zero");
        return double(numrator) / denominator;
}
int main(){
        int n1, n2;
        double result;
        cout<<"Enter two ints(end-of-file ^Z to end):";
        while (cin>>n1>>n2){
                try{
                        result = quotient(n1, n2);
                        cout<< "The quotient is "<<result;
                }catch(invalid_argument ex){    cout<<"invalid_argument:"<<ex.what();
                }catch(logic_error ex){         cout<<"logic_error:"<<ex.what();
                }catch(exception ex){       cout<<"exception:"<<ex.what();
                }catch(...){
                        cout<<"some unexpected exception";
                }
                cout<<"\nEnter two ints(end-of-file ^Z to end):";
        }
        return 0;
}
```

扫描全能王 创建



第14章 输入输出流

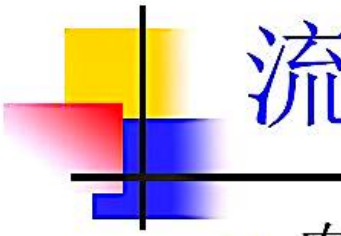
扫描全能王 创建



教学目标

- C++基本流类
- 标准设备输入输出
- 文件流

扫描全能王 创建

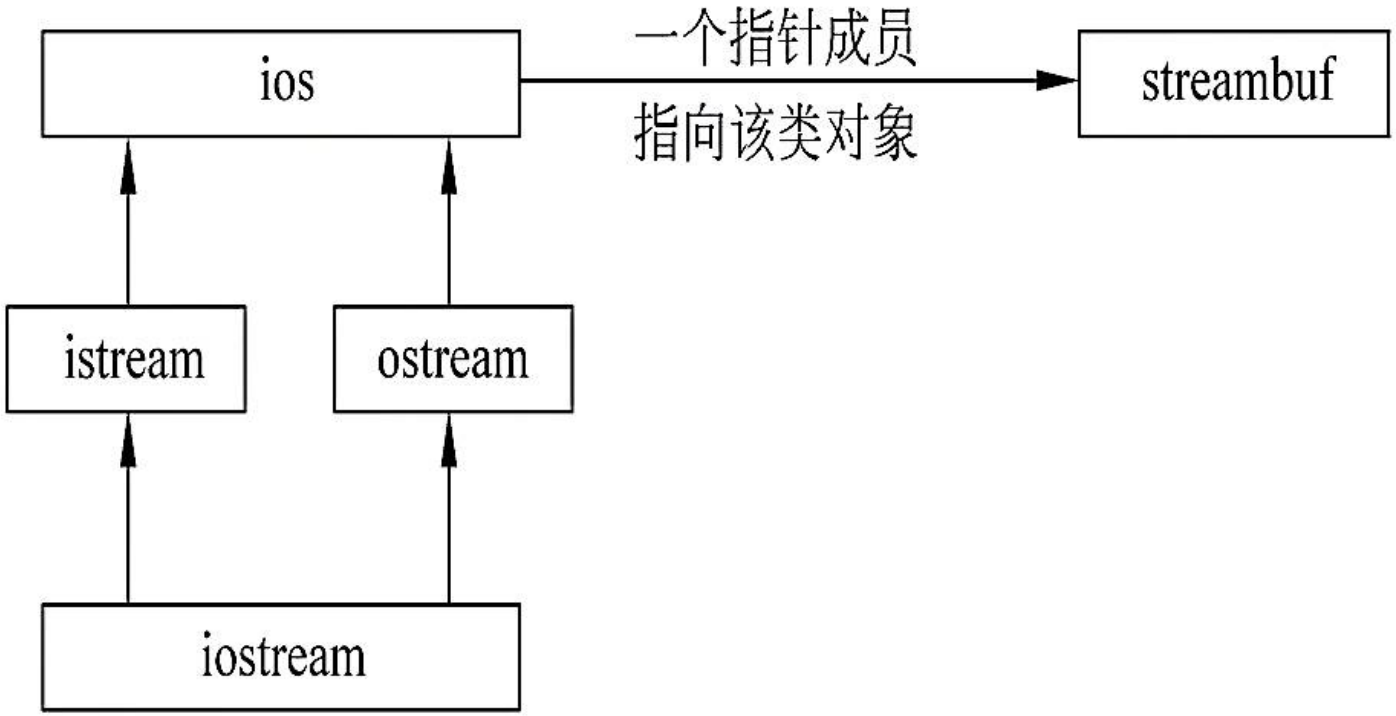


流

- 在C++中，输入输出是通过流来完成的。C++的输出操作将一个对象的状态转换成一个字符序列，输出到某个地方。输入操作也是从某个地方接收到一个字符序列，然后将其转换成一个对象的状态所要求的格式。
- 这看起来很像数据在流动，于是把接收输出数据的地方叫做目标，把输入数据来自的地方叫做源。而输入和输出操作可以看成字符序列在源、目标以及对象之间的流动。
- C++将与输入和输出有关的操作定义为一个类体系，放在一个系统库里，以备用户调用。这个执行输入和输出操作的类体系就叫做流类，提供这个流类实现的系统库就叫做流类库。

扫描全能王 创建

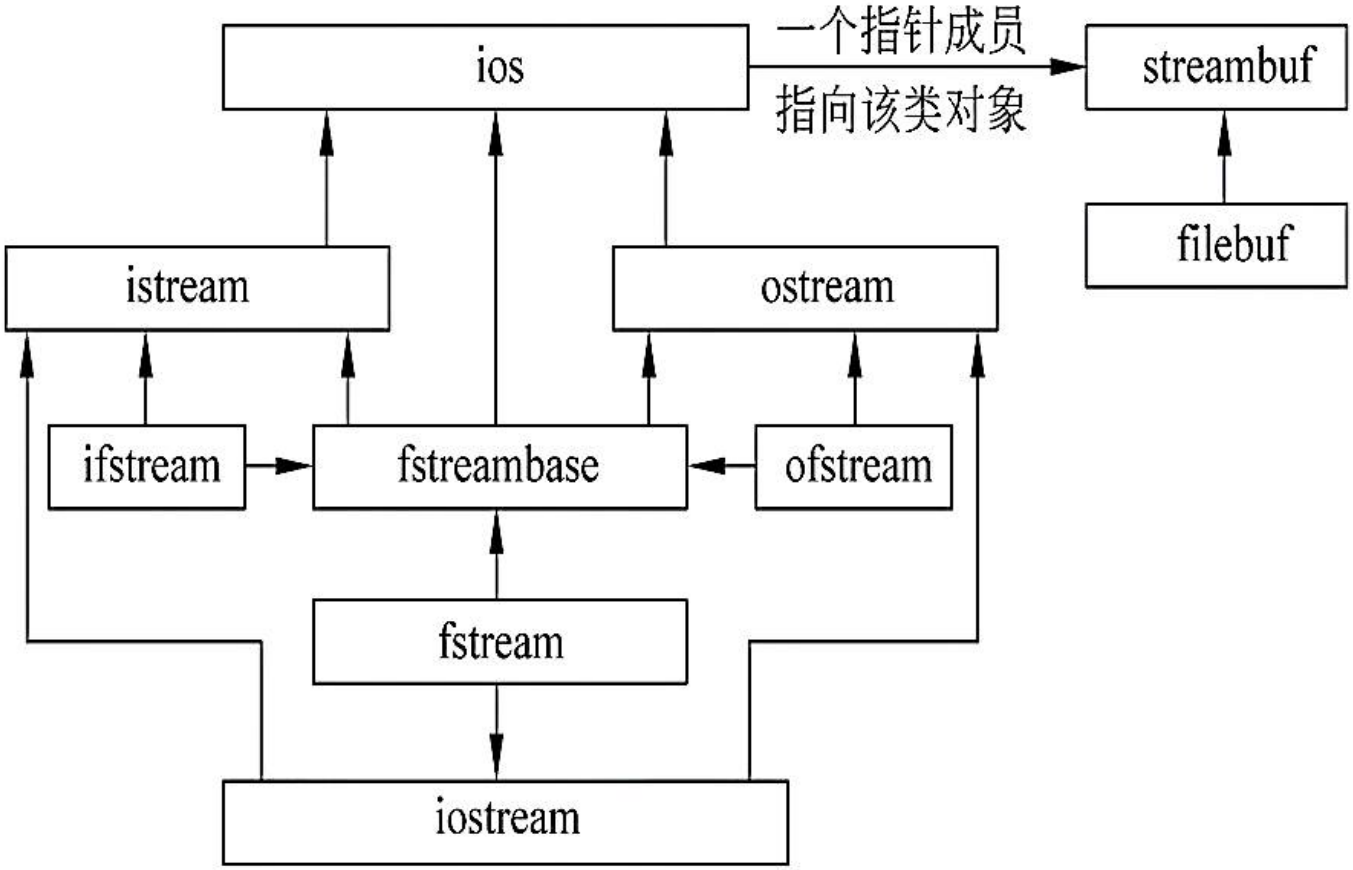
输入/输出流类的继承层次结构



iostream流类库的层次结构

扫描全能王 创建

输入/输出流类的继承层次结构



有文件**I/O**流库的类体系结构

扫描全能王 创建

抽象流基类

	ios	流基类
输入流类	istream	普通输入流类和用于其它输入流的基类
	ifstream	输入文件流类
	istrstream	输入串流类
输出流类	ostream	普通输出流类和用于其它输出流类的基类
	ofstream	输出文件流类
	ostrstream	输出串流类
输入输出流类	iostream	普通输入输出流类和用于其它输入输出流的基类
	fstream	输入输出文件流类
	strstream	输入输出串流类
缓冲流类	streambuf	抽象缓冲流基类
	filefuf	用于磁盘文件的缓冲流类
	strstreambuf	用于串的缓冲流类

扫描全能王 创建

抽象流基类

- ios、istream、ostream和streambuf类构成了C++中iostream输入输出功能的基础。
- iostream类库中，streambuf、ios、istream、ostream、iostream这些基本I/O流类和预定义的cin、cout、cerr和clog在iostream.h文件中说明。
- filebuf、ifstream、ofstream和fstream在fstream.h中说明。
- strstream、istrstream、ostrstream和strstream在strstrea.h中说明。
- 需要注意的是：fstream.h和strstrea.h中都包含了iostream.h，所以如果使用标准输入输出（控制台I/O），只要包含iostream.h头文件即可，如果使用fstream或者strstream，只要包含相应的fstream.h和strstrea.h即可。

扫描全能王 创建

输入输出流

- 头文件iostream中含有4个对象：
  - cin：标准输入流
  - cout：标准输出流
  - cerr：非缓冲标准错误流
  - clog：经缓冲标准错误流
- 格式化I/O：包含头文件iomanip
- 文件处理操作：包含头文件fstream

扫描全能王 创建

输入输出流的成员函数

- 屏幕输出（写操作）
  - 1. 使用预定义的插入符（<<）
  - 2. 使用成员函数put( )写一个字符。
  - 3. 使用成员函数write( )写一个字符串。

扫描全能王 创建

输入输出流的成员函数

- 键盘输入（读操作）
  - 1. 使用预定义的提取符（>>）。
  - 2. 使用成员函数get( )读一个字符。
  - 3. 使用成员函数getline( )读一行字符。
  - 4. 使用成员函数read( )读若干个字符。

扫描全能王 创建

输入输出流的成员函数

- get()：可读取输入的空格；
- getline()：读取一行信息到字符数组中；
- put()：输出一个字符；
- ignore()：跳过制定数量的字符；
- putback()：将一个字符放回到输入流中；
- peek()：返回输入流中的下一个字符，但不将其从输入流中删除；
- read()/write()：无格式输入、输出；
- gcount()：统计输入个数；

扫描全能王 创建

类型要一致，否则出错

```
using namespace std;
#include <iostream>
int  main(void)
{
    int i,j;
    cout<<"输入一个整数（十进制）：";
    cin>>i;
```

256
ff

i=100
j=377

r
?

为变量i提取值时，遇到字符r，把0赋给变量i，输入流中的字符r仍没有处理，但设置了输入错误状态。

当从流中为j提取数据时，发现输入错误状态，不提取数据导致变量j取一个随机值。

i=0
j=31463146314

扫描全能王 创建

输入不正确数据时的处理方法（流的错误处理）

流错误处理成员函数	函数说明
int ios::rdstate() const {return state;}	读取I/O状态字 0表示I/O正常，非0值表示I/O不正常
int ios::bad() const {return state & badbit;}	是否有非法输入/输出操作， 是则badbit位为1 是则函数返回非0值（4），否则返回0
void ios::clear(int _i = 0) {state = _i;)	清除流中的错误，参数缺省值为0
int ios::eof() const {return state & eofbit;}	判断是否到达文件尾， 是返回非零值，否则返回0
int ios::fail() const {return state & (badbit | failbit);}	是否出现I/O操作错误或非法的I/O操作 是则返回非0值，否则返回0
int ios::good() const {return state ==0;}	操作是否正常 是则返回0，否则返回1

扫描全能王 创建

输入不正确的数据时，取空缓冲区中的字符。

```
#include <iostream>
#include <string>
int main() {
    int num;
    std::string buffer; // 用于存储无效输入的缓冲区
    while (true) {
        std::cout << "请输入一个整数: ";
        std::cin >> num;
        if (std::cin.good()) {
            // 输入有效，退出循环
            break;
        } else {
            // 输入无效，清除错误状态并丢弃无效输入
            std::cin.clear(); // 清除错误标志
            std::getline(std::cin, buffer); // 丢弃无效输入
            std::cout << "输入无效，请重新输入一个整数" << std::endl;
        }
    }
    std::cout << "您输入的整数是: " << num << std::endl;
    return 0;
}
```

扫描全能王 创建

输入操作的成员函数

- get()：可读取输入的空格；
- getline()：读取一行信息到字符数组中。

1. 用get函数提取字符或字符串时，要单独提取换行符。//提取时遇到换行符结束，但是换行符仍在输入流的缓冲区中。
2. 用getline提取字符串时，当实际提取的字符个数小于第二个参数指定的字符个数时，则将字符串结束符也提取出来，但不保存。
3. EOF表示文件结束符，在iostream.h中定义为－1。使用Ctrl+Z键输入。

扫描全能王 创建

提取字符或字符串的成员函数

int istream::get()
istream& istream::get(char &);
istream& istream::get(unsigned char &);
istream& istream::get(signed char &);
istream& istream::get(char *,int,char ='\n');
istream& istream::get(unsigned char *,int,char ='\n');
istream& istream::get(signed char *,int,char ='\n');
istream& istream::getline(char *,int,char ='\n');
istream& istream::getline(unsigned char *,int,char ='\n');
istream& istream::getline(signed char *,int,char ='\n');

扫描全能王 创建

从输入流中读取字符和字符串

```
using namespace std;
#include <iostream>
int   main(void )
{
   char  c1,c2,c3, str1[80],str2[100];
    cout << "输入三个字符： ";
    c1=cin.get();
    cin.get(c2);  cin.get(c3);
    cin.get();                    //A
    cout << "输入第一行字符串： ";
    cin.get(str1,80);
    cin.get();                          //B
    cout <<"输入第二行字符串： ";
    cin.getline(str2,80);                          //D
   cout <<"c1="<<c1<<'\t' <<"c2="<<c2<<'\t' <<"c3="<<c3<<'\n';
    cout<< "str1="<<str1<<"\nstr2="<<str2<<'\n';
}
                                                                    ;
```

```
输入三个字符： VC+
输入第一行字符串： visual C++ programming
输入第二行字符串： language test input
c1=V    c2=C    c3=+
str1=visual C++ programming
str2=language test input
```

扫描全能王 创建

使用函数gcount和ignore

```
using namespace std;
#include <iostream>
int   main(void )
{
    char c1, str1[100];
    int num;
    cout<<"输入一个字符： ";
    cin.get(c1);
    cin.ignore(80,'\n');
    cout << "输入第一行字符串： ";
    cin.getline(str1,80);
    num=cin.gcount();
    cout <<"c1="<<c1<<'\n';
    cout<< "str1="<<str1<<"\n";
    cout <<“num=”<<num<<'\n';
//读取一行字符串后在字符串数组中 插入一个空格。
}
```

```
输入一个字符： abcd Abort
输入第一行字符串： computer depart.
c1=a
str1=computer depart.
num=17
Press any key to continue
```

扫描全能王 创建

输出操作的成员函数

- put()：输出一个字符；
  - ostream& ostream::put(char)
  - ostream& ostream::put(unsigned char)
  - ostream& ostream::put(signed char)
- flush()：刷新一个输出流。
  - ostream& ostream::flush()

```
cout.put('H').put('e').put('l').put('o').put('\n');
```


扫描全能王 创建

重载提取和插入运算符（实现对象的输入和输出）

例：从键盘输入两个复数相加。

```
using namepace std;
#include <iostream>
class Complex{
 double Real,Image;
public:
Complex(double a=0,double b=0)
{ Real=a; Image=b;}

Complex operator +(Complex &s)
 { Complex t;
   t.Real=Real+s.Real;
   t.Image=Image+s.Image;
  return t;
};
void Show()
{  cout<<Real<<' '<<Image<<endl;}
void SetR(double a) {  Real=a;  }
void SetI(double a) {Image=a;}
};
```

```
int main()
{
  Complex a,b,c;
  double x,y;
  cin>>x>>y;
  a.SetR(x);
  a.SetI(y);
  cin>>x>>y;
  b.SetR(x);
  b.SetI(y);
  c=a+b;
  c.Show();
}
```

cin>>a;

cin>>b;

cout<<c;

控制台输入：

1 2 <CR>

2 3 <CR>

控制台输出：

3 5

扫描全能王 创建

>>(提取)和<<(插入)的重载：

格式：friend istream &operator>>(istream &,ClassName &);

friend ostream &operator<<(ostream &,ClassName &);

```
using namespace std;
#include <iostream>
class Complex{
 double Real,Image;
public:
Complex(double a=0,double b=0)
{ Real=a; Image=b;}

Complex operator+(Complex &s)
{  Complex t;
   t.Real=Real+s.Real;
   t.Image=Image+s.Image;
   return t;
}
friend istream &operator>>(istream &,Complex &);
friend ostream &operator<<(ostream &,Complex &);
};//Complex 定义

istream &operator>>(istream &is,Complex &x)
{  is>>x.Real>>x.Image;
   return is;}
```

```
ostream &operator<<(ostream &os,Complex &x)
{
    os<<x.Real;
    if(x.Image>0) cout<<'+';
     os<<x.Image<<'i';
     return os;}
            int main()
              {
               Complex a,b,c;
               cin>>a>>b;
               c=a+b;
               cout<<"a="<<a<<' '<<"b="<<b<<endl;
               cout<<"c="<<c<<endl;
                }
```

控制台输入：

1 2 3 -4 <CR>

控制台输出：

a=1+2i b=3-4i

c=4-2i

扫描全能王 创建



```
#include<iostream>
class RMB{
    int yuan, jiao, fen;
public:
    RMB(int y=0, int j=0, int f=0){
        float ff = float(y*100.0 + j*10.0 +
        yuan = (int)ff;
        jiao = int(ff * 10) % 10;
        fen = int(ff * 100) % 10;
    }
    RMB(float f){                                   //单参构造函数
        yuan = (int)f;
        jiao = int(f * 10) % 10;
        fen = int(f * 100) % 10;
    }
    operator float(){        //转换函数，转为float
        return float(yuan*100.0 + jiao*10.0 + fen)/100;
    }
    friend istream &operator>>(istream & is, RMB &m);
    friend ostream &operator<<(ostream & os, RMB &m);
};
```

```
istream &operator>>(istream & is, RMB &m){
    cout<<"输入元 角 分\n";
    int yuan,jiao,fen;
    is>>yuan>>jiao>>fen;
    m=RMB(yuan,jiao,fen);
    return is;
}

ostream &operator<<(ostream & os, RMB &m){
    os<<m.yuan<<"元"<<m.jiao<<"角"<<m.fen<<"分";
    return os;
}

int main(){
    RMB b2;
    cin>>b2;
    cout<<b2<<endl;
}
```

扫描全能王 创建

格式化I/O

1. 使用状态标志和成员函数进行格式化输出

（1） 控制格式的标志位。

- 定义标志位的组合（静态变量）：

  basefield□ del|oct|hex

  adjustfield□ left|right|internal

  floatfield□ seientific|fixed

扫描全能王 创建



格式化I/O

（2） 设置标志的成员函数

long setf()：设置流格式标志

例：

```
cout.setf(ios::uppercase|ios::scientific);
```

long unsetf(long):清除流格式标志

扫描全能王 创建

格式化I/O

(3) 控制输出格式和成员函数

int width( )

设置域宽;

int fill( )

填充字符;

int precision( )

设置精度;

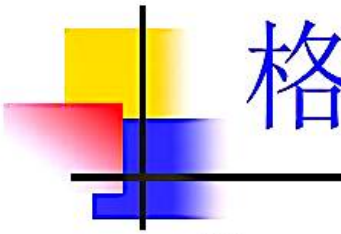
```
#include <iostream>
using namespace std;
int main()
{
        cout.precision (4);
        cout.width(10);
        cout<<10.12345<<endl;
        cout.fill ('*');
        cout.width (10);
        cout<<10.12345<<endl;
        cout<<"Hi"<<endl;
        cout.width (10);
        cout.setf(ios::left);
        cout<<10.12345<<endl;
        return 0;
}
```

扫描全能王 创建

```
The number is 185
The number is b9
The number is 0xb9
The number is       0xb9
The number is 0xb9
The number is 0x       b9
The number is 0x       b9
```

```
using namespace std;
#include <iostream>
#include <iomanip>
int main()
{
    const int number = 185;
    cout<<"The number is "<<number<<endl;
    cout<<"The number is "<<hex<<number<<endl;
    cout.setf(ios::showbase);    //showbase表示：在十六进制数前加上0x
    cout<<"The number is "<<hex<<number<<endl;
    cout<<"The number is ";
    cout.width(10);
    cout<<hex<<number<<endl;
    cout<<"The number is ";
    cout.width(10);
    cout.setf(ios::left);       //setf()用于进行格式设置
    cout<<hex<<number<<endl;
    cout<<"The number is ";
    cout.width(10);
    cout.setf(ios::internal);//在指定任何引导标志或基之后填充字符
    cout<<hex<<number<<endl;
    cout<<"The number is "<<setw(10)<<hex<<number<<endl;
    return 0;
}
```


扫描全能王 创建



格式化I/O

2. 使用流操作符进行格式输出

- 系统I/O流类库中所定义的操作符被放在iomanip文件中。

扫描全能王 创建

常量名	含 义
skipws	跳过输入中的空白
left	输出数据按输出域左边对齐输出
right	输出数据按输出域右边对齐输出
internal	在指定任何引导标志或基之后填充字符
dec	转换基数为十进制形式
oct	转换基数为八进制形式
hex	转换基数为十六进制形式
showbase	输出带有一个表示制式的字符
showpoint	浮点输出时必须带有一个小数点和尾部的0
uppercase	十六进制数值输出使用大写A~F，科学计数，显示使用大写字母E
showpos	在正数前添加一个“+”号
fixed	使用定点形式表示浮点数
scientific	使用科学计数法表示浮点数
untibuf	每次插入之后,ostream::osfx刷新该流的缓冲区。默认缓冲单元为cerr
setbase(base)	将基数设置为base

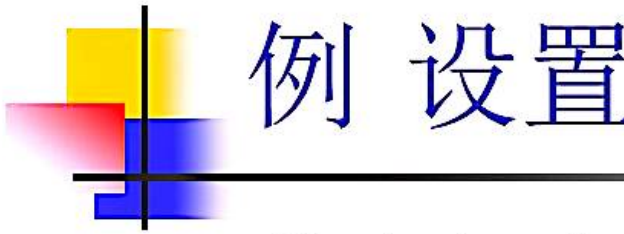
扫描全能王 创建

例 设置整数流的基数

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{   int n;
    cout << "Enter a decimal number: ";
    cin >> n;
    cout << n << " in hexadecimal is: "
         << hex << n << '\n'
         << dec << n << " in octal is: "
         << oct << n << '\n'
         << setbase( 10 ) << n << " in decimal is: "
         << n << endl;
    return 0;
}
```

```
Enter a decimal number: 10
10 in hexadecimal is: a
10 in octal is: 12
10 in decimal is: 10
```


扫描全能王 创建



例 设置浮点数精度

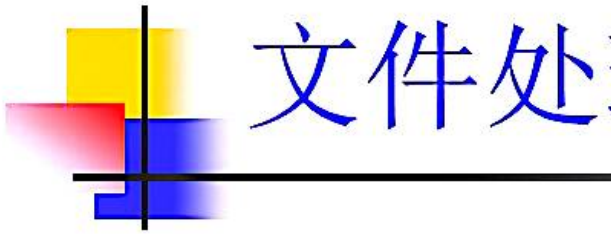
```
#include <iostream>
#include <iomanip>
#include <cmath>
using namespace std;
int main()
{  double root2 = sqrt( 2.0 );
   int places;
   cout << setiosflags( ios::fixed)
   << "Precision set by the precision member function:"
   << endl;
```


扫描全能王 创建

```
    for ( places = 0; places <= 4; places++ )
    {     cout.precision( places );
          cout << root2 << '\n';}
    cout << "Precision set by the " << "setprecision
    manipulator:\n";
    for ( places = 0; places <= 4; places++ )
          cout << setprecision( places ) << root2 << '\n';
    return 0;
}
```

```
Precision set by the precision member functior
1
1.4
1.41
1.414
1.4142
Precision set by the setprecision manipulator:
1
1.4
1.41
1.414
1.4142
```


扫描全能王 创建

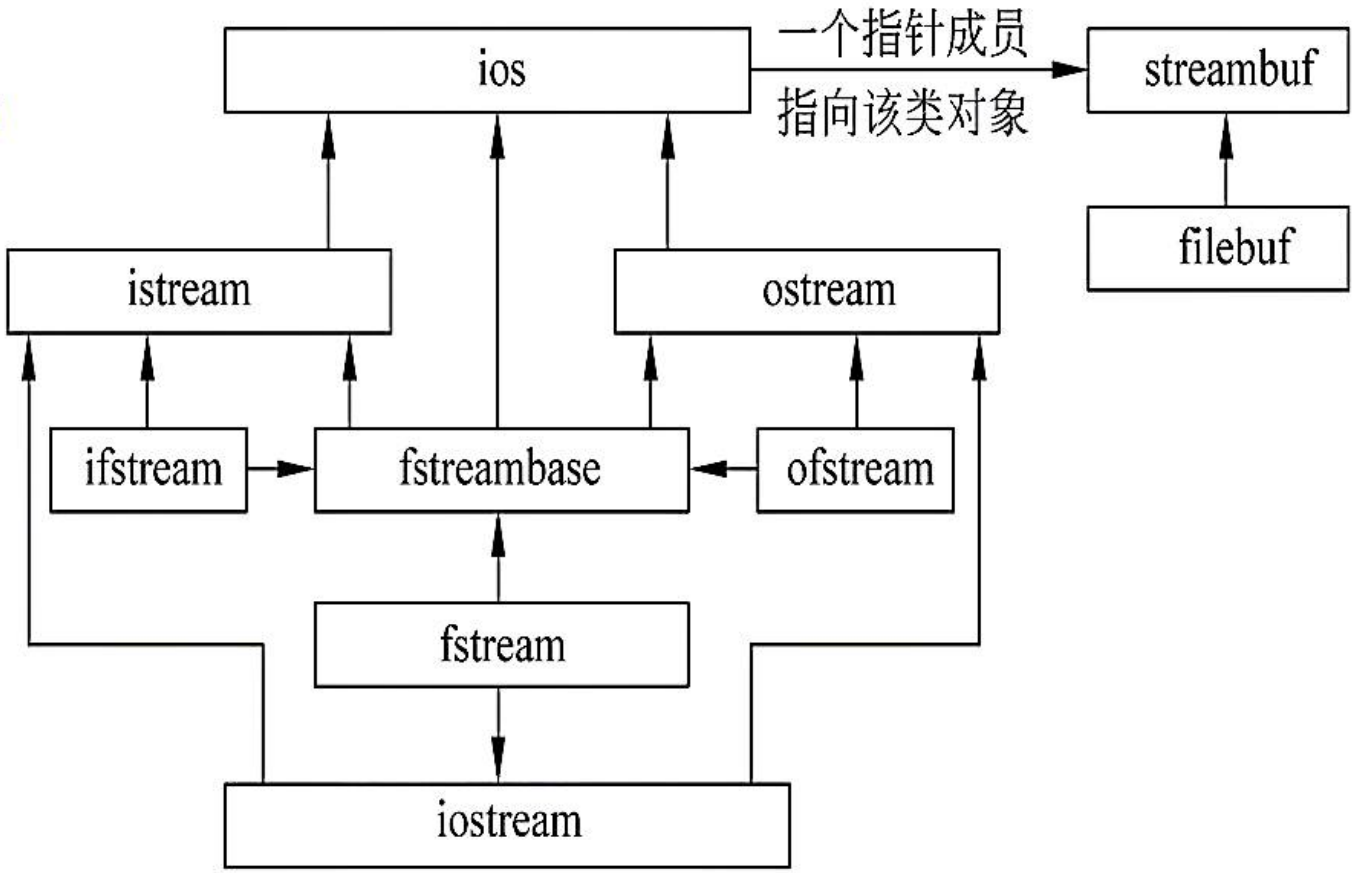


文件处理

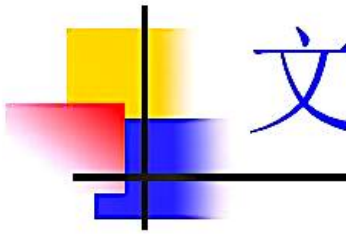
- 基本概念
  - 打开和关闭文件
  - 读
  - 写
  - 文件指针
  - 缓冲区

扫描全能王 创建

文件处理



扫描全能王 创建



文件处理

1. 磁盘文件打开关闭操作

（1） 打开文件的两种方法

方法一：

fstream <流对象名>;

<流对象名>.open("<文件名>",<方式>);

或者

fstream <流对象名>.open("<文件名>“,<方式>)

扫描全能王 创建

文件处理

方法二：

① 打开写文件

ofstream <流对象名>;

<流对象名>.open(" <文件名> ");

或者

ofstream <流对象名>.open(" <文件名> ");

扫描全能王 创建



文件处理

② 打开读文件

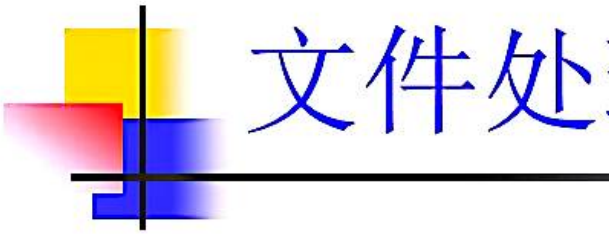
ifstream<流对象名>;

<流对象名>.open("<文件名>");

或者

ifstream <流对象名>.open("<文件名>");

扫描全能王 创建



文件处理

（2） 关闭文件方法

〈流对象名〉.close( );

扫描全能王 创建

打开文件的成员函数

- **void ifstream ::open(const char*, int = ios::in, int = filebuf::openprot);**
- **void ofstream::open(const char*, int = ios::out, int = filebuf::openprot);**
- **void fstream ::open(const char*, int , int = filebuf::openprot);**

打开方式	说明
ios::in	打开文件进行读操作，这种方式可避免删除现存文件的内容
ios::out	打开文件进行写操作，这是默认模式
ios::ate	打开一个已有输入或输出文件并查找到文件尾
ios::app	打开文件以便在文件的尾部添加数据
ios::nocreate	如果文件不存在，则打开操作失败
ios::noreplace	如果设置了**ios :: ate**或**ios::app**，则可打开已有文件，否则不能打开
ios::binary	指定文件以二进制方式打开，默认为文本方式
ios::trunc	如文件存在，将其长度截断为零并清除原有内容

也可把多个枚举量进行"或"运算，例如

```
file.open( "rm.txt", ios_base::out | ios_base::trunc );
```

如果不明确指定**binary**二进制文件，都是文本文件。

扫描全能王 创建

文件的打开与关闭

每次打开文件之后，都应及时判断是否成功。若成功，对象非0，若失败，对象为0。例如，

```
ifstream  f1("file.dat");
if (!f1) {     //判断打开文件是否成功
   cout <<"不能打开输入文件："<<"file.dat"<<'\n';
   exit(1);
}
```

扫描全能王 创建

打开文件进行读写（注意换行符的提取）

```
#include <fstream>
#include <iostream>
using namespace std;
int main(void)
{
    char fileName[80];
    char buffer[255];
    cout<<"File name: ";
    cin>>fileName;
    ofstream fout(fileName);
    fout<<"This line written directly to the file...\n";
    cout<<"Enter text for the file: ";
    cin.ignore(1,'\n'); //接受输入文件名后的回车
    cin.getline(buffer,255);
    fout<<buffer<<"\n";
    fout.close();
    ifstream fin(fileName);
    cout<<"Here's the contents of the file:\n";
    char ch;
    while(fin.get(ch))
        cout<<ch;
    cout<<"\n***End of file contents.***\n";
    fin.close();
    return 0;
}
```

```
File name: test1
Enter text for the file: This text is written to the file!
Here's the contents of the file:
This line written directly to the file...
This text is written to the file!

***End of file contents.***
```


扫描全能王 创建

在文件末尾添加内容

```
#include <fstream>
#include <iostream>
using namespace std;
int main(void){
    char fileName[80];
    char buffer[255];
    cout<<"Please reenter the file name: ";
    cin>>fileName;
    ifstream fin(fileName);
    if(fin)          {
        cout<<"Current file contents: \n";
        char ch;
        while(fin.get(ch))
                cout<<ch;
        cout<<"\n***End of file contents. ***\n";        }
    fin.close();
    cout<<"\nOpening "<<fileName<<" in append mode...\n";
```


扫描全能王 创建

```
ofstream fout(fileName,ios::app);
if(!fout)       {cout<<"Unable to open "<<fileName<<" for
appending.\n";
        return(1);          }
cout<<"\nEnter text for the file: ";
cin.ignore(1,'\n');
cin.getline(buffer,255);
fout<<buffer<<"\n";
fout.close();
fin.open(fileName);
if(!fin)
{
        cout<<"Unable to open "<<fileName<<" for reading.\n";
        return(2);          }
cout<<"\nHere's the contents of the file: \n";
char ch;
while(fin.get(ch))
        cout<<ch;
cout<<"\n***End of file contents.***\n";
fin.close();
return 0;}
```

```
Please reenter the file name: test1
Current file contents:
This line written directly to the file...


***End of file contents. ***

Opening test1 in append mode...

Enter text for the file: More text for the file

Here's the contents of the file:
This line written directly to the file...

More text for the file!

***End of file contents.***
```

扫描全能王 创建

文本文件的操作

- 文本文件是将所有内容都保存为文本，如**54321**这样的数字被保存为一串字符，效率低。但优点是诸如**DOS**命令和**Windows**命令行程序可直接阅读文本。

用分隔符分割的文本，如**cpp,h,txt,html**等

- 可用**>>/<<**进行读/写
- 可用**get(ch)/put(ch)**进行读/写
- 混合方式，如**getline(buf, 300)/outfile<<buf**

如何判断读到文本文件尾？

读到**0**，或**ios::eof()**返回为**1**。如：

```
while(infile>>ch)
    outfile<<ch;
```


扫描全能王 创建

使用构造函数打开文件，并把源程序文件拷贝到目的文件中。应用>>和<<

```
#include <iostream>
#include <fstream>
#include <cstdlib>
using namespace std;
int  main(void ){
     char  filename1[256],filename2[256];
     cout <<"输入源文件名： ";
     cin  >>filename1;
     cout <<"输入目的文件名： ";
     cin  >>filename2;
     ifstream  infile(filename1,ios::in|ios::nocreate);
     ofstream  outfile(filename2);
     if (!infile ) {
       cout << "不能打开输入文件："<<filename1<<'\n';
       exit(1);}
     if (!outfile ) {
       cout << “不能打开目的文件："<<filename2<<'\n';
       exit(2);}
     char ch;
     infile.unsetf(ios::skipws);  //不要跳过文件中的分隔字符
     while (infile >> ch)      outfile <<ch;
     infile.close();     outfile.close();
}
```

1）文件若不提供全通路名，则对当前操作

2）在文件无数据可读时，返回0

扫描全能王 创建

使用构造函数打开文件，使用成员函数实现文件的拷贝。应用**get**和**put**

```
#include <iostream>
#include <fstream>
#include <cstdlib>
using namespace std;
int  main(void )
{
    char  filename1[256],filename2[256];
    cout <<"输入源文件名： ";
    cin  >>filename1;
    cout <<"输入目的文件名： ";
    cin  >>filename2;
    ifstream  infile(filename1,ios::in|ios::nocreate);
    ofstream  outfile(filename2);
    if (!infile ) {
      cout << "不能打开输入文件： "<<filename1<<'\n';
      exit(1);}
    if (!outfile ) {
      cout << “不能打开目的文件： "<<filename2<<'\n';
      exit(2);}
    char ch;
    while (infile.get(ch))    outfile.put(ch);
     infile.close();     outfile.close();
}
```


扫描全能王 创建

使用成员函数打开文件，实现文件的拷贝。应用**getline**和**<<**

```
#include <iostream>
#include <fstream>
#include <cstdlib>
using namespace std;
int  main(void ){
    char  filename1[256],filename2[256];
    char  buff[300];
    cout <<"输入源文件名： ";
    cin  >>filename1;
    cout <<"输入目的文件名： ";
    cin  >>filename2;
    fstream infile,  outfile;
    infile.open(filename1,ios::in|ios::nocreate);
    outfile.open(filename2,ios::out);
    if (!infile ) {
      cout << "不能打开输入文件： "<<filename1<<'\n';
      exit(1);}
    if (!outfile ) {
      cout << “不能打开目的文件： "<<filename2<<'\n';
      exit(2);}
    while (infile.getline(buff,300))   outfile<<buff<<'\n';
     infile.close();     outfile.close();
}
```


扫描全能王 创建

设文本文件**data.txt**中有若干个实数，每一个实数之间用分隔符分开。求出文件中的这些实数的平均值。

```
#include <iostream>
#include <fstream>
using namespace std;
int main(void)
{
        ifstream infile("data.txt", ios::in|ios::nocreate);
        if(!infile)
        {
                cout<<"不能打开输入文件：\n";
                exit(1);
        }
        float sum = 0, temp;
        int count = 0;
        while(infile >> temp){
                sum += temp;
                count++;
        }
        cout<<"平均值= "<<sum/count<<"\t"<<"count= "<<count<<'\n';
        infile.close();
}
```


扫描全能王 创建

二进制文件的操作

- 在使用>>运算符读取文本文件时，会发生一些字符转换，空白字符被忽略。如果想要阻止进行字符转换，必须打开一个以二进制方式访问的文件.

扫描全能王 创建

二进制文件的操作

下面以将内存中存放数组a的内容写入data.dat文件为例。a在VC中占20个字节，用len表示。

① 使用ofstream构造函数中的模式参量指定二进制输出模式:

```
ofstream wf("data.dat",ios::binary);
wf.write((char *)a, len);
```

② 使用ofstream构造一个输出流，然后使用open函数指定写入方式:

```
ofstream wf;
wf.open("data.dat",ios::binary);
wf.write((char *)a, sizeof(a));
```

不能用是否读到0来判断到文件尾，只能用ios::eof()函数：

```
while(!infile.eof){...}
```


扫描全能王 创建

产生一个二进制数据文件，将1～500之间的所有偶数写入文件data.dat中。

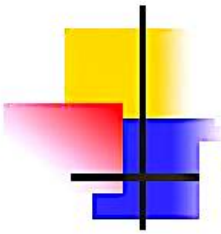
```
#include <iostream>
#include <fstream>
#include <cstdlib>
using namespace std;
int main(void)
{
    ofstream outfile("data.dat",ios::out|ios::binary);
    if(!outfile)
    {
        cout<<"不能打开目的文件data.dat\n";
        exit(1);
    }
    for(int i = 2; i < 500; i += 2)
        outfile.write((char *)&i,sizeof(int));
    outfile.close();
}
```


扫描全能王 创建

从产生的数据文件data.dat中读取二进制数据，并在显示器上按每行10个数的形式显示。

```
#include <iostream>
#include <fstream>
#include <cstdlib>
using namespace std;
int main(void){
    ifstream infile("data.dat", ios::in|ios::binary|ios::nocreate);
    if(!infile)
    {
        cout<<"不能打开目的文件data.dat\n";
        exit(1);
    }
    int i,a[250];
    infile.read((char *)a,sizeof(int)*250);
    for(i = 0; i < 250; i++)
    {
        cout<<a[i]<<'\t';
        if((i+1)%10 == 0) cout<<'\n';
    }
    cout<<'\n';
    infile.close();
}
```


扫描全能王 创建



使用成员函数read和write来实现文件的拷贝

```
#include <iostream>
#include <fstream>
#include <cstdlib>
using namespace std;
int main(void)
{
    char filename1[256],filename2[256];
    char buff[4096];
    cout<<"输入源文件名：";
    cin>>filename1;
    cout<<"输入目的文件名：";
    cin>>filename2;
    fstream infile,outfile;
    infile.open(filename1,ios::in|ios::binary|ios::nocreate);
    outfile.open(filename2,ios::out|ios::binary);
```


扫描全能王 创建

```
if(!infile)
    {
        cout<<"不能打开输入文件："<<filename1<<'\n';
        exit(1);
    }
    if(!outfile)
    {
        cout<<"不能打开目的文件："<<filename2<<'\n';
        exit(2);
    }
    int n;
    while(!infile.eof())
    {
        infile.read(buff,4096);
        n = infile.gcount();
        cout<<n;
        outfile.write(buff,n);
    }
    infile.close();
    outfile.close();
}
```

扫描全能王 创建

将对象的内容写入二进制文件

```
#include <fstream>
using namespace std;
class Animal
{
public:
    Animal(int weight, long days):itsWeight(weight),
    DaysAlive(days){}
    ~Animal(){}
    int GetWeight()const{return itsWeight;}
    void SetWeight(int weight){itsWeight = weight;}
    long GetDaysAlive()const{return DaysAlive;}
    void SetDaysAlive(long days){DaysAlive = days;}
private:
    int itsWeight;
    long DaysAlive;
};
```


扫描全能王 创建

```
int main()
{
    char fileName[80];
    cout<<"Please enter the file name: ";
    cin>>fileName;
    ofstream fout(fileName, ios::binary);
    if(!fout)
    {
        cout<<"Unable to open "<<fileName<<" for writing.\n";
        return(1);
    }
    Animal Bear(50,100);
    fout.write((char*)&Bear,sizeof(Bear));
    fout.close();

    ifstream fin(fileName, ios::binary);
    if(!fin)
    {
        cout<<"Unable to open "<<fileName<<" for reading.\n";
        return(1);
    }
```


扫描全能王 创建



```
    Animal BearTwo(1,1);

    cout<<"BearTwo weight: " <<BearTwo.GetWeight()<<endl;
    cout<<"BearTwo days: "<<BearTwo.GetDaysAlive()<<endl;

    fin.read((char *) &BearTwo,sizeof(BearTwo));

    cout<<"BearTwo weight: " <<BearTwo.GetWeight()<<endl;
    cout<<"BearTwo days: "<<BearTwo.GetDaysAlive()<<endl;
    fin.close();
    return 0;
}
```


扫描全能王 创建

随机文件操作

- C++不仅可以顺序访问文件，还可以随机访问。为实现对文件的随机访问，I/O流类库提供了定位读指针和定位写指针的成员函数。
- 文件指针是什么？
  一个文件作为数据序列，移动文件指针到指定位置，再读写。
  一般是从头到尾移动，也可按指令移动。

```
enum seek_dir{
    beg = 0,      //文件开始处作为参考点(缺省)
    cur = 1,      //文件当前位置作为参考点
    end = 2       //文件结束处作为参考点
};
```


扫描全能王 创建

文件的随机访问

函数	说明
basic_istream& seekg(streampos pos);	将输入流的get指针定位到pos绝对位置
basic_istream& seekg(streamoff off, ios_base::seek_dir dir);	将输入流的get指针定位到off相对位置，相对于dir
streampos tellg();	返回输入流的当前定位
basic_ostream& seekp(streampos pos);	将输出流的put指针定位到pos绝对位置
basic_ostream& seekp(streamoff off, ios_base::seek_dir dir);	将输出流的put指针定位到off相对位置，相对于dir
streampos tellp();	返回输出流的当前定位

扫描全能王 创建

产生一个5~1000之间的奇数文件（二进制文件），将文件中的第21~30之间的数依次读出并输出。

```
using namespace std;
#include <fstream>
#include <iostream>
int main(void)
{
    ofstream outfile("data.dat", ios::out|ios::binary);
    if(!outfile)
    {
        cout<<"不能打开目的文件data.dat\n";
        exit(1);
    }
    int i;
    for(i = 5; i < 1000; i += 2) outfile.write((char*)&i,sizeof(int));
    outfile.close();
```


扫描全能王 创建

45 47 49 51 53 55 57 59 61 63

```
    ifstream f1("data.dat",ios::in|ios::binary);
    if(!f1)
    {
        cout<<"不能打开目的文件data.dat\n";
        exit(1);
    }
    int x;
    f1.seekg(20 * sizeof(int));
    for(i = 0; i < 10; i++)
    {
        f1.read((char*)&x,sizeof(int));
        cout<<x<<'\t';
    }
    f1.close();
}
```


扫描全能王 创建

实例编程：电话簿管理程序

模拟电视频道的存储和选择。创建一个文件，通过键盘输入电视频道序号和名称，并写入文件；文件中的内容可以输出到屏幕上。

```
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;
struct TVChannel
{   int channelNum;
    char channelName[20];
};
void outputLine( ostream &output, const TVChannel &tv )
{   output << setiosflags( ios::left ) << setw( 10 ) << tv.channelNum ;
    output << setw( 20 ) << tv.channelName << '\n';
}
```


扫描全能王 创建

```
int main()
{    ofstream outTV( "tv.dat", ios::ate | ios::binary);
     if ( !outTV ) { cerr << "File could not be opened." << endl;
                     return 1;}
     cout << "Enter channel number " << "(1 to 100, 0 to end input)\n? ";
     TVChannel tv;
     cin >> tv.channelNum;
     while ( tv.channelNum > 0 && tv.channelNum <= 100 ) {
          cout << "Enter Channel Name\n? ";
          cin >> tv.channelName;
          outTV.seekp((tv.channelNum-1)*sizeof(TVChannel));
          outTV.write( (char *)( &tv ), sizeof( TVChannel ) );
          cout << "Enter channel number\n? ";
          cin >> tv.channelNum;          }
     outTV.close();
     ifstream inTV( "tv.dat", ios::in );
     if ( !inTV ) {  cerr << "File could not be opened." << endl;
                     return 1;}
    cout<<setiosflags(ios::left)<<setw(10)<<"ChannelNum"<<setw(20)<<"Channel Name"<<endl;
     inTV.read( (char *)( &tv ), sizeof( TVChannel ) );
     while ( inTV&& !inTV.eof() ) {
          if ( tv.channelNum != 0 ) outputLine( cout, tv );
          inTV.read( (char *)( &tv ),      sizeof( TVChannel ) );   }
     return 0;
}
```

扫描全能王 创建